Python Engineering - Classes & Objects Q&A;

Where the class and methods live, what self means, and where a calculated variable lives

1. Which module should contain the class?

For EX21, place the complete `class VerticalState` definition inside `vertical_state.py`.

```
ex21_vertical_state_class/
|-- vertical_state.py
`-- main.py
```

vertical_state.py defines **what a VerticalState is**. main.py creates and uses objects.

2. Where do the methods go?

Methods that belong to VerticalState are written **inside the class**, so they also live in vertical_state.py.

```
# vertical_state.py

import numpy as np

class VerticalState:
    def __init__(self, h, v, t=0.0, g=9.81):
        self.h = h
        self.v = v
        self.t = t
        self.g = g

    def specific_energy(self):
        ...

    def state_vector(self):
        ...

    def euler_step(self, dt):
        ...
```

Indentation matters: a function indented inside the class is a method of that class.

3. Is a method basically a function?

Yes. The simplest mental model is: **a method is a function defined inside a class**. It normally receives the current object through self.

```
# normal function
def specific_energy(h, v, g):
    return g*h + 0.5*v**2

# method
def specific_energy(self):
    return self.g*self.h + 0.5*self.v**2
```

The normal function needs h, v, and g passed in. The method can read them from the object.

4. Can we calculate e and return e?

Yes. This is perfectly valid and often clearer:

```
def specific_energy(self):
    e = self.g * self.h + 0.5 * self.v**2
    return e
```

e is a **local method variable**. It exists during the method call, but it is not automatically stored as object state.

5. Where is e stored?

```
earth_energy = earth_state.specific_energy()
```

The returned value is stored in earth_energy by the calling code. It is not stored inside earth_state unless we explicitly create an attribute.

```
def specific_energy(self):
    self.e = self.g * self.h + 0.5 * self.v**2
    return self.e
```

This would store self.e on the object, but it is not recommended here: if h or v changes later, self.e may become stale.

6. What should we do with a derived quantity such as energy?

When a quantity is cheap to recompute from the current state, calculating it on demand is usually safer:

```
def specific_energy(self):
    e = self.g * self.h + 0.5 * self.v**2
    return e
```

This guarantees the energy matches the current h, v, and g.

7. Why use self.h instead of h?

Because specific_energy() needs the height **stored on the current object**. There is no local variable h inside that method unless we explicitly pass one.

```
def specific_energy(self):
    # self.h comes from the object
    return self.g * self.h + 0.5 * self.v**2
```

If earth_state.h=100, self.h is 100 for earth_state. If moon_state.h=250, self.h is 250 for moon_state.

8. Why not pass h, v, and g again?

We could, but that defeats part of the purpose of storing state in the object and can create contradictory values.

```
earth_state.h = 100.0

# confusing:
earth_state.specific_energy(h=500.0, v=20.0, g=9.81)
```

Use self.h when the value belongs to object state.

9. Why is self in the method definition?

Python needs a reference to the concrete object. When we write:

```
earth_state.specific_energy()
```

conceptually Python behaves like:

```
VerticalState.specific_energy(earth_state)
```

earth_state enters as the first argument and is named self inside the method.

10. Why don't we type self when calling the method?

Because Python passes it automatically when the method is called through an object.

```
def euler_step(self, dt):
    ...

earth_state.euler_step(0.1)

# conceptually:
VerticalState.euler_step(earth_state, 0.1)
```

11. Which names use self. and which do not?

Element	Example	Why
Persistent object attribute	self.h	Part of object state
Another persistent attribute	self.g	May differ between objects
Temporary method parameter	dt	Needed only for this call
Local calculated variable	e	Temporary unless explicitly stored

12. Recommended EX21 structure

```
# vertical_state.py
import numpy as np

class VerticalState:
    def __init__(self, h, v, t=0.0, g=9.81):
        self.h = h
        self.v = v
        self.t = t
        self.g = g

    def specific_energy(self):
        e = self.g*self.h + 0.5*self.v**2
        return e

    def state_vector(self):
        return np.array([self.h, self.v, self.t])

    def is_ascending(self):
        return self.v > 0.0

    def euler_step(self, dt):
        self.h = self.h + self.v*dt
        self.v = self.v - self.g*dt
        self.t = self.t + dt

# main.py
from vertical_state import VerticalState

def main():
    earth_state = VerticalState(100.0, 20.0)
    energy = earth_state.specific_energy()
    earth_state.euler_step(0.1)
    print(energy)
    print(earth_state.state_vector())

if __name__ == '__main__':
    main()
```

Final summary

vertical_state.py contains the class and its methods.
main.py creates and uses objects.
A **method** is a class-associated function.
self represents the current object.
self.h is stored state; **h** may only be a temporary parameter.
A local variable such as **e** can be returned without being stored on the object.
Using **self.e** would store it persistently, but it may become stale if the state changes.